Module 5 — Agents and Orchestration

This is the agents-and-orchestration module of a knowledge base on production enterprise AI; it can be understood on its own.

Why this module exists. Everything so far — prompting, structure, tools, retrieval, structured knowledge — becomes an *agent* when you put the model in a **loop** where it can reason, act, observe, and decide what to do next toward a goal. Agents are where enterprise AI moves from "answers questions" to "does work." They're also where risk, cost, and unpredictability spike. This module covers how agents work, how to architect and orchestrate them, and how to keep them safe and observable. The Systems-of-Record theme continues: agents are the interaction and orchestration layer that reads and, carefully, writes to Systems of Record.

This module moves through eight topics. First, the agent loop, which runs from perception through reasoning to action. Second, single-agent versus multi-agent architectures. Third, orchestration frameworks such as LangGraph and CrewAI. Fourth, planning. Fifth, state and memory management. Sixth, guardrails. Seventh, human-in-the-loop. And eighth, observability.

5.1 The agent loop: from perception through reasoning to action

Why it matters. An **agent** is a large language model operating in a **loop**: it perceives a situation, reasons about what to do, takes an action — usually a tool call — observes the result, and repeats until the goal is met or it stops. The distinguishing feature compared with a plain model call is **autonomy over control flow**. The model decides *what to do next and when it's done*, rather than following a fixed script. This is the generalization of the ReAct pattern introduced earlier in the prompting material.

An analogy. A single model call is a vending machine: input in, output out. An agent is a junior analyst you give a goal to, such as "prepare the ACME renewal quote." They look things up, use tools, hit obstacles, adjust, and come back when done — exercising judgment about the steps.

How it works — the loop. First, **perception**: take in the goal, context, conversation, and latest observations, meaning retrieved data and tool results. Second, **reasoning and decision**: the model decides the next action — call a tool, ask the user, or finish. This is the ReAct-style movement from thought to action. Third, **action**: execute a tool call, such as querying a System of Record, running code, searching, or writing a record — governed by *your* code. Fourth, **observation**: feed the result back into context. Fifth, **loop** until a stopping condition — goal met, max steps, error, or human handoff.

Key design choices. Stopping criteria, so you avoid infinite loops by capping iterations or budget; how much autonomy you allow versus a fixed workflow; what tools are available, which defines the action

space; and how errors are handled, whether by retry, replan, or escalate.

The spectrum of "agentic." At one end are **workflows**: fixed, code-defined sequences with model steps — predictable and testable. At the other end are **autonomous agents**: the model plans its own path — flexible but less predictable. A widely-shared industry view, for example Anthropic's "Building Effective Agents," is to **prefer the simplest thing that works** — often a structured workflow, not a fully autonomous agent. Add autonomy only when the task's variability demands it.

Where it goes wrong. These pitfalls are expanded throughout the module. Runaway loops drive cost and latency. **Compounding errors** mean each step's mistakes propagate. There's tool misuse, and unpredictability that defeats testing. Autonomy is a cost, not a virtue. On the compounding-error math: the oft-quoted claim that "a 95-percent-reliable step run ten times is about 60-percent reliable" is just zero-point-nine-five to the tenth power, which is about sixty percent. It's a useful illustration but a **worst-case** model that assumes steps are **independent and non-recoverable**. Real systems break that assumption on purpose. **Verification steps, retries, self-correction, and human-in-the-loop** raise the *effective* per-step reliability, which is precisely *why* those mechanisms matter. So treat the scary number as motivation for guardrails, not as a fixed law.

A concrete example. Consider a "renewal prep" agent. It perceives the goal plus the account, reasons that it needs the current contract, then calls the CRM, observes the terms, checks open support issues, drafts a renewal summary, and asks the rep to confirm before writing anything back. It's a loop, with a human gate on the write.

5.2 Single-agent versus multi-agent architectures

Why it matters. You can build with **one agent** — one model loop with many tools — or with **multiple agents**: specialized agents that collaborate, for example a planner, researchers, a writer, and a critic. Multi-agent is fashionable and sometimes powerful, but it adds coordination cost and failure surface. Choosing correctly is a major architectural decision.

How it works. A **single-agent** design is one loop, one context, and a toolbox. It's simpler, easier to debug and evaluate, and cheaper. It scales surprisingly far. Its limits: too many tools degrade selection, and very long tasks strain one context.

There are several **multi-agent patterns**. The **orchestrator-and-worker** pattern, also called supervisor, has a lead agent decompose the task and delegate to sub-agents, then synthesize. It's common and effective. A **role-based crew** has agents with personas or roles collaborate — researcher, writer, reviewer. A **sequential pipeline** feeds the output of one into the next, though this is often better modeled as a workflow. And

debate or critique has agents critique each other to improve quality, which overlaps with using a model as a judge.

Why multi-agent can help: separation of concerns, parallelism such as fan-out research, specialized prompts and tools per role, and larger effective context, since each sub-agent has its own window. Anthropic's multi-agent research system and similar efforts report gains on **broad, parallelizable search and research** tasks.

Why it often hurts: coordination overhead, error propagation between agents, higher cost and latency from many model calls, harder debugging, and **context-passing loss** where agents miscommunicate. A large body of practitioner experience says **most "multi-agent" needs are actually one good agent or a workflow.**

The public debate in 2025 that crystallized the consensus. Cognition's *"Don't Build Multi-Agents"* argued that parallel subagents produce conflicting outputs without shared context, and coined the term "context engineering." Anthropic's *"How we built our multi-agent research system"* argued that an orchestrator plus parallel subagents is essential for broad research. The synthesis both sides converged on: multi-agent wins for **parallelizable read and research fan-out under a single orchestrator**, not for tightly-coupled tasks with shared decisions. Even Cognition later shipped a coordinator-of-agents, converging on the orchestrator model.

When to use it (and when not to). Start single-agent, or with a workflow. Reach for multi-agent when the task **parallelizes** into independent subtasks, such as researching many sources at once; when it needs **genuinely distinct specializations or tools**; when it exceeds one context's practical limit; or when it benefits from **independent critique**. Prefer **orchestrator-and-worker** over free-for-all agent swarms, and constrain communication paths.

On **maturity**: multi-agent orchestration is **stabilizing** — there are real wins in specific shapes, such as parallel research and clear role separation, but it's still immature for tightly-coupled tasks where coordination cost dominates.

Where it goes wrong. Multi-agent as a solution in search of a problem. Unbounded agent-to-agent chatter, causing cost explosion. Lost context between agents. No clear termination. Debugging a distributed, nondeterministic system. And diffusion of responsibility, where no single place enforces guardrails.

A concrete example. A market-research task fans out five sub-agents, one per competitor, in parallel, each retrieving and summarizing, with an orchestrator synthesizing — a good multi-agent fit. By contrast, a configure-price-quote flow is better as a **single agent plus workflow with gated steps**, because the steps are tightly coupled and each needs governance, not parallelism.

5.3 Orchestration frameworks

Why it matters. Orchestration frameworks manage the agent loop, tool calling, state, multi-step control flow, and — for some — multi-agent coordination, so you don't rebuild that plumbing. Choosing one shapes how you build, test, and operate. They differ mainly in **how much structure and control** they impose versus how much they abstract away.

The main options — frameworks and how they differ. A note to verify: this landscape churns fast. Nearly every framework below hit a 1.0 or general-availability milestone, or was renamed or merged, in late 2025 through mid 2026. Verify current names and versions.

LangGraph, in the LangChain ecosystem, models agents as **graphs or state machines**: nodes are steps, edges are control flow, with explicit **state**, cycles, checkpoints, human-in-the-loop pauses, and durable execution. LangGraph and LangChain both reached 1.0, general availability, in October 2025 — their first stable releases, with breaking changes deferred until a future 2.0. The prebuilt ReAct agent now lives in the LangChain agents module. It's favored for **production** because control flow is explicit and inspectable. It pairs with **LangSmith** for observability and evaluation.

CrewAI is a **role-based multi-agent** framework, with agents that have a role, goal, and backstory, plus tasks and processes. CrewAI 1.0 reached general availability, with **Flows** for event-driven orchestration and **AMP** as the enterprise control plane. It's fast to prototype and more opinionated.

Microsoft Agent Framework is **the merger of AutoGen and Semantic Kernel**, in public preview in October 2025 with general availability around early 2026, for .NET and Python. It combines AutoGen's multi-agent abstractions with Semantic Kernel's enterprise features — state, telemetry, middleware — plus graph workflows. AutoGen and Semantic Kernel are now maintenance-only; new work goes here.

OpenAI Agents SDK plus AgentKit. The Agents SDK reached general availability in March 2025 as the successor to Swarm. AgentKit, announced at DevDay in October 2025, adds a visual **Agent Builder**, the **ChatKit** embeddable UI, a **Connector Registry**, and **Evals for Agents**.

Google ADK, the Agent Development Kit, is an open-source multi-agent framework for Python, Java, Go, and TypeScript, with workflow and dynamic-routing agents. It powers Google's own products.

LlamaIndex is retrieval- and RAG-centric, with agent and workflow features. It's strong when data and RAG are the core.

Pydantic AI is a type-safe framework in the Pydantic ecosystem, reaching 1.0 in late 2025. It's popular for typed, testable agents.

Cloud and platform-native options. AWS Bedrock AgentCore reached general availability in October 2025, covering the full lifecycle: runtime, session isolation, memory, identity, observability, and gateway.

Google Gemini Enterprise Agent Platform is the rebrand of Vertex AI Agent Builder, with Agentspace merged in; its Agent Engine and Memory Bank are generally available. **Microsoft Foundry Agent Service** is generally available — "Azure AI Foundry" was renamed **Microsoft Foundry**. There are also **application-platform-native** agent layers, such as **Salesforce Agentforce** and **ServiceNow AI Agents**, which trade openness for deep System-of-Record integration and built-in governance.

Model-native "agentic" APIs. Vendors increasingly offer built-in tool-use loops, computer-use, and memory primitives, reducing framework dependence.

When to use it (and when not to). If you need production-grade control, observability, and durability, reach for LangGraph or a cloud-native equivalent with explicit state and checkpoints. For a fast multi-agent prototype, reach for CrewAI, Microsoft Agent Framework in Python, or Google ADK. For typed, testable agents, reach for Pydantic AI. For RAG-heavy work, reach for LlamaIndex. For deep System-of-Record integration with built-in governance, reach for platform-native options such as Agentforce or ServiceNow, weighing the walled-garden versus composable tradeoff.

Compare on the four axes that actually decide a governed-write deployment, not on "vibe." First, the **control-flow model** — explicit graph or state-machine, as in LangGraph, versus conversation-driven versus role-crew. Second, **durability and resumability** — can a run checkpoint and resume for audit and recovery, which is LangGraph's strong suit? Third, **native human-in-the-loop** — first-class pause-for-approval-then-resume. Fourth, **observability and evaluation integration** — built-in tracing and evaluation hooks. For anything touching money, records, or customers, weight the first three heavily.

A general principle: favor frameworks that make **control flow and state explicit** for anything that touches money, records, or customers. Avoid lock-in where you can. **MCP**, for the agent-to-tool boundary, and **A2A**, for the agent-to-agent boundary, both help keep the tool and inter-agent layers portable.

A caveat: this space churns fast, and frameworks rise and fall quarterly. Bet on **portable concepts** — loops, state, tools, checkpoints, evaluations — over any one framework, and keep the model and tool boundaries clean.

Where it goes wrong. Framework lock-in. Over-abstracted "magic" that's impossible to debug or evaluate. Adopting multi-agent frameworks for single-agent problems. Version churn. And hiding the security boundary inside framework internals you don't control.

A concrete example. A regulated insurer builds its claims-triage agent on **LangGraph** so that every state transition is explicit, checkpointed — meaning resumable and auditable — and can **pause for human approval** before any payout write. It then exposes System-of-Record tools via **MCP** so the same governed tools serve other agents.

5.4 Planning

Why it matters. Planning is how an agent decides the *sequence* of steps to reach a goal, rather than reacting one step at a time. Good planning is what lets agents handle complex, multi-step tasks. Poor planning is a top cause of agents wandering, looping, or missing steps.

The main options — approaches. ReAct, or implicit planning, plans as you go, one step at a time. It's flexible and adaptive but can lose the thread on long tasks. **Plan-and-execute** generates a full plan first, then executes steps, re-planning as needed. It's more structured, better for known-shaped tasks, and the plan is inspectable and approvable. **Decomposition** breaks the goal into sub-goals or sub-tasks — least-to-most, or task trees. It pairs with the orchestrator-and-worker pattern. **Reflection, or self-critique**, has the agent review its own progress or output and revise, in the Reflexion style. It improves quality at a cost. **Tree or graph search over actions** explores multiple paths; it's more research-oriented and expensive. **Deterministic scaffolding** encodes the plan as a **workflow** when the steps are known, letting the model fill in the reasoning within fixed structure. This is often the most reliable enterprise choice.

When to use it (and when not to). For a known, repeatable process, **encode the plan as a workflow** — predictable, testable, governable. For a variable or open-ended task, let the agent plan, using ReAct or plan-and-execute, with strict stopping conditions. Add reflection for quality-critical outputs. **The more autonomy in planning, the more you need evaluation, guardrails, and human gates.**

Where it goes wrong. Over-planning trivial tasks. Plans that don't adapt when reality diverges, because there's no re-planning. No termination, causing loops. Plans that skip validation or permission steps. And the model **confidently planning a wrong approach** and executing it, which compounds errors.

A concrete example. A configure-price-quote agent uses a **deterministic plan** — requirements, then configure, then price, then quote — because the process is known and each stage has governance. Within each stage, the model reasons freely, for example about which clarifying questions to ask, but it cannot reorder or skip stages. Structure where it matters, flexibility where it helps.

5.5 State and memory management

Why it matters. Agents need to **remember** — within a task, meaning working state, and, increasingly, across sessions, meaning persistent memory. But the model itself is **stateless**: each call only knows what's in the context window. Memory is therefore an **engineered system around the model**, not a model feature. Getting it right is central to coherent, personalized, non-repetitive agents. Getting it wrong causes context bloat, forgetting, and cost blowups. The cutting edge of *persistent cross-session* memory is covered later in the knowledge base.

How it works — the memory taxonomy. Short-term, or working, memory is the current context window: conversation, recent tool results, scratchpad. It's bounded and must be curated. **Context management**: because the window is finite, you **summarize or compress** older turns, **trim** irrelevant content, and keep critical facts pinned. "Context engineering" — deciding what's in the window at each step — is now a core skill. **Long-term memory** is persisted outside the model and **retrieved when relevant**; this is RAG applied to memory. It comes in three kinds. **Episodic** memory covers past interactions and events, for example "last quarter we discussed X." **Semantic** memory covers facts and preferences about the user or domain, for example "this customer requires NET-60 terms." **Procedural** memory covers learned how-to and skills. **Storage** options include a vector store for semantic recall, a key-value or profile store for structured facts and preferences, and increasingly the **System of Record itself** as durable memory — write agreed facts back to the CRM, and don't invent a shadow store. **State, for workflows and agents**, means explicit state objects, as in LangGraph, with checkpoints for durability, resumption, and audit.

The three hard mechanics. This is where "add memory" actually lives. First, **what to remember — the write or salience policy.** You don't persist everything. Trigger a durable write on an **explicit user statement**, such as "always bill us NET-60"; a **repeated** fact; or an agent inference **above a confidence gate**. A cheap "extractor" step decides what's worth keeping and discards the rest — otherwise memory bloats and drifts. Second, **conflict resolution, when memory contradicts the System of Record.** The rule "the System of Record wins" needs a mechanism: mark agent-derived memories as **provisional**, and on use **re-fetch the authoritative value from the System of Record** rather than trusting the cached memory — write-through or read-through. Stale or provisional memory never overrides a live System-of-Record read. Third, **when to retrieve it — the retrieval trigger.** There are two options with different cost and quality. **Always-on injection** puts relevant memories in every prompt — simple, but it bloats context and can mislead. **Tool-invoked recall** has the agent explicitly call a "recall memory" tool when it needs it — more controlled, and it avoids polluting context. Tool-invoked recall is usually the better default for enterprise agents.

When to use it (and when not to). Use **working memory plus summarization** for single sessions. Add **long-term memory** when cross-session continuity or personalization has clear value. **Prefer the System of Record as the durable memory of record** for business facts — a customer's terms belong in the CRM, not only in an agent's private memory. This keeps one source of truth and respects entitlements. Use vector memory for softer recall, such as past conversations.

Where it goes wrong. Context bloat — stuffing in all history drives cost and latency up and causes "lost in the middle," which drives quality down. **Memory poisoning** — a wrong or injected "fact" persists and corrupts future behavior, a security concern for persistent memory. **Stale or contradictory memory** — remembered facts conflict with the System of Record; the System of Record must win. **Privacy** — persisting personally identifiable information in agent memory creates governance and retention

obligations, and cross-user memory leakage is a breach. **Shadow source of truth** — an agent memory that diverges from the System of Record.

A concrete example. A customer-success agent keeps working memory of the current chat, summarizes long threads to stay within budget, and stores durable facts — such as "prefers quarterly reviews" or an escalation contact — **as structured fields on the CRM account**. So the next session, and human reps, see the same truth, entitlements apply, and nothing lives in an ungoverned side-store.

5.6 Guardrails

Why it matters. Guardrails are the controls that keep an agent within safe, correct, compliant bounds — on **inputs**, blocking malicious or off-topic requests; on **outputs**, preventing PII leakage, toxicity, and hallucinated commitments; and on **actions**, constraining what tools it can call and with what authority. For enterprises, guardrails are the difference between a demo and something you can point at customers, money, and records. **The central principle: the model is never the security boundary** — it can be manipulated — so guardrails live in deterministic code and policy around it.

How it works — the layers. Input guardrails detect and deflect prompt injection, jailbreaks, off-scope requests, and disallowed topics, and validate and parse inputs. **Output guardrails** handle PII and secret detection and redaction, toxicity and safety filters, format and schema validation, groundedness checks — meaning the answer is supported by sources — and **policy checks**, such as "never promise a discount" or "always include disclosure." **Action guardrails**, which are most important for agents, include several controls. **Least-privilege tools** mean the agent can only call what it needs, with **read versus write separation**. **Permission enforcement in code** means the tool layer checks the *user's* entitlements on every call — the model can request, but code authorizes. **Confirmation and approval gates** cover high-stakes actions. **Idempotency, transaction limits, rate limits, and spend caps** bound the blast radius. And **allow-lists and sandboxing** cover code execution, external calls, and MCP servers. **Deterministic plus model-based** defense combines hard rules — regex, policy engines, schema — with model-based classifiers, where a small model flags unsafe content. That's defense in depth.

Tools. NVIDIA NeMo Guardrails, Guardrails AI, Llama Guard and Prompt Guard, vendor safety filters and moderation endpoints, policy engines such as OPA, DLP tools, plus platform-native guardrails in Agentforce and ServiceNow.

When to use it (and when not to). Scale guardrails to **stakes and autonomy**. An internal brainstorming bot needs little. An agent that writes quotes or touches customer data needs the full stack, with **writes and high-stakes actions always gated**. Enforce entitlements and destructive-action limits in **code**, not prompts.

Where it goes wrong. Trusting the prompt — "I told it not to" — is bypassed by injection. Enforce in code. **Over-blocking** — heavy-handed filters frustrate users and break legitimate flows. **Guardrails only on inputs and outputs, not actions** — the dangerous gap for agents. **No injection defense in RAG or agents** — retrieved or tool content carries hidden instructions. **Confused-deputy** — the agent acts with its own broad privileges on behalf of a manipulated request; propagate user identity and apply least privilege.

A concrete example. A configure-price-quote agent may *read* the catalog and *draft* a quote freely, but the tool that **writes a quote or applies a discount** enforces the rep's entitlements, discount limits by role, mandatory approval above a threshold, idempotency keys so there are no duplicate quotes on retry, and a full audit log — all in code, independent of anything the model "decided."

5.7 Human-in-the-loop

Why it matters. Human-in-the-loop inserts human judgment at critical points — approval before consequential actions, review of uncertain outputs, escalation of hard cases. It's the pragmatic bridge between "fully manual" and "fully autonomous," and it's how enterprises deploy agents responsibly *today* for anything high-stakes. It also builds the trust and the training data needed to safely expand autonomy over time.

How it works — the patterns. Approve-or-reject gates: the agent proposes, and a human approves before the action commits. This is the default for writes to a System of Record, payments, and external communications. It requires the framework to **pause and resume**, as with LangGraph checkpoints. **Confidence-based escalation**: auto-handle high-confidence cases and route low-confidence ones to humans. A caveat: token-level logprobs are often unavailable on reasoning-model APIs and are a poor proxy for answer correctness. Prefer verifier or judge scoring, self-consistency agreement, groundedness checks, or explicit uncertainty elicitation. **Human-on-the-loop, or oversight**: the agent acts autonomously but humans monitor and can intervene or override. This suits lower-stakes or reversible actions. **Review-and-edit**: the agent drafts, and a human edits and sends — email, quote, summary. **Feedback capture**: human decisions become **evaluation and training data**, improving the system and justifying more autonomy over time.

When to use it (and when not to) — where to put the human. Gate on **stakes, reversibility, and confidence**. High-stakes and irreversible actions — pricing, contracts, payouts, external customer commitments — need **mandatory approval**. Reversible or low-stakes actions can use oversight, or run autonomously with logging. Uncertain cases escalate. Design the human step to be **fast and well-contextualized** — show the proposed action, rationale, and sources — so it doesn't become a bottleneck.

Where it goes wrong. Rubber-stamping — humans approve without reading, from automation bias. Mitigate with clear, concise, well-justified proposals and spot-checks. **Bottlenecks** — too many gates kill

the efficiency you deployed AI for. Gate only what matters. **No audit trail** of who approved what, which is a compliance gap. **Approval fatigue** leading to gate removal without re-assessing risk.

A concrete example. A renewal agent drafts the quote and shows the rep the proposed line items, price, the discount applied, the policy it cites, and flagged risks. The rep approves with one click, or edits — and *then* the write commits to the System of Record. Every approval is logged with user, timestamp, and the exact proposal.

5.8 Observability

Why it matters. Observability is the ability to see what an agent did and why — every prompt, tool call, retrieval, decision, token, dollar, and latency — in development and production. Agents are **nondeterministic, multi-step, and expensive**. Without observability you can't debug failures, control cost, catch regressions, prove compliance, or improve the system. It's not optional at enterprise scale. This is the runtime half of LLMOps.

How it works — what to capture. Tracing is the full execution trace of a run: each model call, with prompt, response, tokens, latency, cost, and model version; each tool call, with inputs, outputs, and errors; each retrieval, with query, chunks, and scores; state transitions; and the final outcome. **Distributed tracing** across multi-agent and multi-step flows now uses the **maturing and published OpenTelemetry GenAI semantic conventions**, which standardize agent-invocation and MCP tool-call spans. Major observability tools — LangSmith, Langfuse, Arize Phoenix — now ingest OTLP natively, making the layer swappable. **Metrics** cover latency, both per step and end-to-end; tokens and cost per run; tool success and error rates; retrieval quality signals; task-completion rate; human-intervention rate; and guardrail triggers. **Evaluation integration** runs online evaluations on production traffic — sample and score, and alert on drift. **Logging for audit** keeps immutable records of actions taken, especially writes, of approvals, and of the data and permissions involved. **Feedback loops** — thumbs up or down, corrections, escalations — feed evaluation sets and improvement.

Tools. LangSmith, Arize Phoenix, Langfuse, Helicone, Weights and Biases (Weave), Datadog LLM Observability, the OpenTelemetry GenAI conventions, plus platform-native monitoring in Agentforce, ServiceNow, and cloud agent services.

When to use it (and when not to). Instrument **from day one** — retrofitting observability is painful. Prioritize **tracing, cost, and audit logging** for any production agent, and add online evaluation and drift detection as usage grows. For regulated or high-stakes agents, treat **audit logging of actions and approvals** as a hard requirement.

Where it goes wrong. With no tracing, "the agent did something weird" is unsolvable. With no cost tracking, you get surprise bills from deep loops and verbose contexts. Logging sensitive data insecurely is a risk, since traces contain PII and secrets — govern them. Metrics without evaluations mean you see latency but not *quality* drift. And no audit trail for actions is a compliance failure.

A concrete example. When a customer disputes a quote the agent generated, ops pulls the full trace: the requirements captured, the catalog queried with entitlements, the price rule applied, the discount and its approver, the exact model version, and the timestamped write to the System of Record — a complete, auditable reconstruction. That same trace pipeline flags a week-long rise in "needs-clarification" outcomes, surfacing a catalog data issue before it becomes a revenue problem.

Module 5 takeaways

- An **agent** is a model in a **loop** with autonomy over control flow. Prefer the **simplest thing that works** — often a workflow, not full autonomy.
- Use **single-agent designs or workflows** first. Go **multi-agent** only for parallelizable or genuinely specialized work, choosing orchestrator-and-worker over swarms. Multi-agent is **stabilizing**, not a default.
- On **frameworks**: LangGraph 1.0 for control and production; CrewAI, Microsoft Agent Framework, or Google ADK for multi-agent; Pydantic AI for typed agents; LlamaIndex for RAG; cloud services such as Bedrock AgentCore, Gemini Enterprise Agent Platform, and Microsoft Foundry; and platform-native options for deep System-of-Record integration. The landscape churns fast, so bet on **portable concepts** and keep the tool and inter-agent layers portable with MCP and A2A.
- On **planning**: encode known processes as workflows, and let agents plan open-ended tasks with strict stopping conditions.
- **Memory** is engineered around a stateless model. Prefer the **System of Record as durable memory of record** for business facts.
- On **guardrails**: the model is **never** the security boundary. Enforce entitlements and action limits in code, and gate writes and high-stakes actions.
- Apply **human-in-the-loop** based on stakes, reversibility, and confidence. Make the human step fast and well-contextualized, and capture decisions as training and evaluation data.
- **Observability** — tracing, cost, audit logging, online evaluation — is mandatory for production agents. Instrument from day one.